



Use Case 2

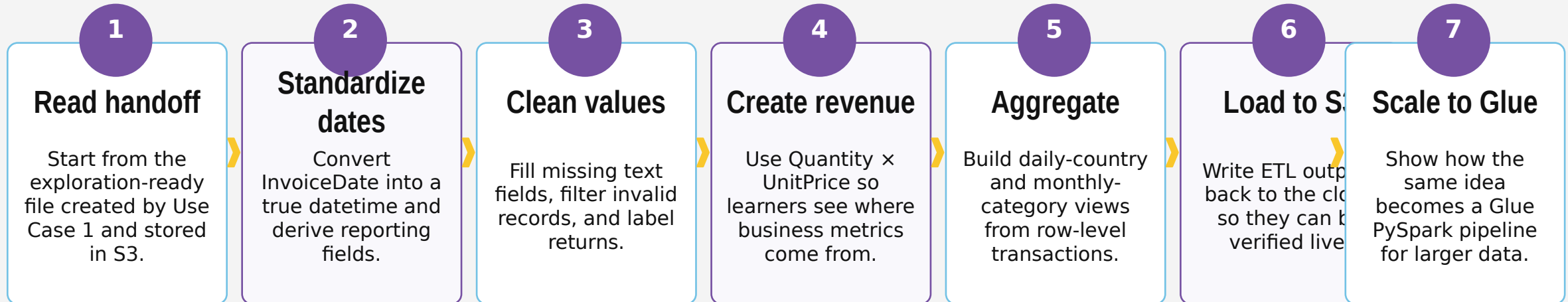
Retail ETL Pipeline for Analytics

**BITS learner deck • S3 + SageMaker Studio + AWS
Glue + Spark + Redshift pattern**

Goal: help learners clean the handoff file, create revenue-based outputs, and understand how the same logic moves from notebook code into Glue PySpark.

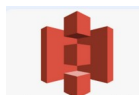
Use Case 2 — learner step map

Learner roadmap — follow these named stages while you code live



Use Case 2 — AWS technologies and why they fit

From notebook logic to scalable ETL: each service has a simple job in the story



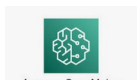
Amazon S3

Amazon S3

Shared cloud storage for input, cleaned data, and ETL outputs.

Bucket → object → key keeps the path structure simple.

Why used here: All steps read from or write to S3 so there is one source of truth for the class.

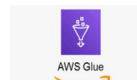


Amazon SageMaker

Amazon SageMaker

Notebook environment for the version of ETL. Great for showing each transformation step before scaling it.

Why used here: Learners can see the business logic clearly before seeing Glue.



AWS Glue

AWS Glue

Managed ETL service for production-style Spark processing.

You supply logic; Glue manages the runtime and job execution.

Why used here: Useful when the same ETL flow must be repeatable and scalable.



PySpark

PySpark / Spark

Distributed engine used under Glue.

RDD / DataFrame split work across a cluster and recover from failures.

Why used here: Advantage: faster processing and fault tolerance for larger datasets.



amazon REDSHIFT

Redshift

Warehouse pattern for downstream analytics.

Structured place where curated outputs can later be loaded.

Why used here: Included as the “where this output can land next” architecture step.

Why not stop at a file script?

A local script is fine for practice. In AWS, S3 + Glue make the ETL sharable, repeatable, and ready to scale beyond one machine.

Spark in one line

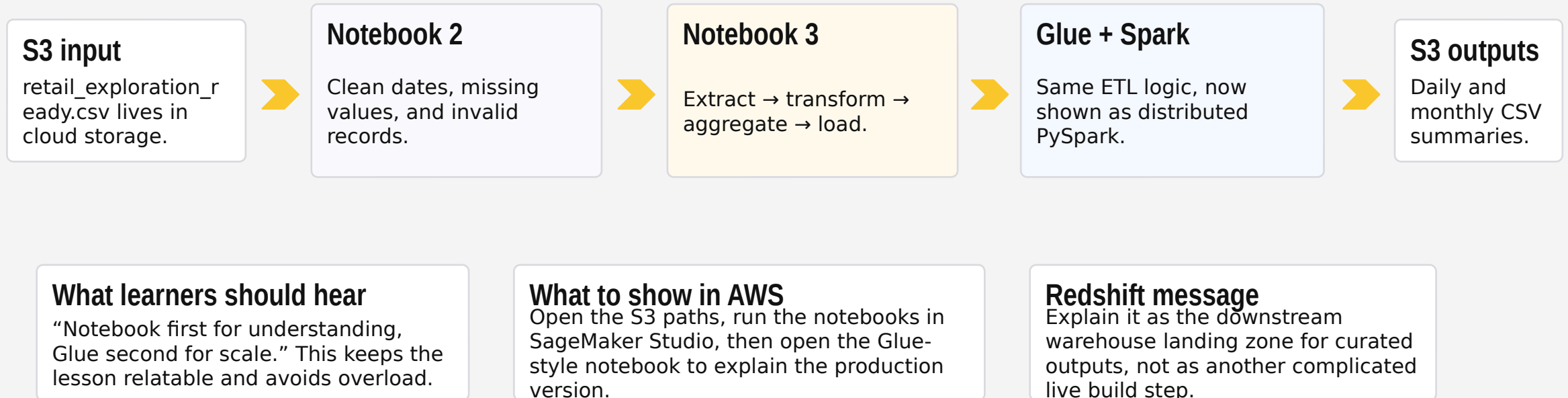
Spark breaks large work into smaller pieces and runs them in parallel. RDD = resilient distributed dataset, designed for distributed, fault-tolerant processing.

Glue in one line

Glue is the managed ETL wrapper around Spark. It removes server setup so the class can focus on the transformation logic.

Use Case 2 — service demonstration flow

Show the class where each service appears in the ETL journey



Use Case 2 — input fields become ETL outputs



Help learners see how raw columns become reporting-ready tables

Field	What learners should notice	Why it matters in the lab
InvoiceDate	Starts as text in the handoff file.	Converted to datetime, then split into date and month fields.
Description	Free-text product description.	Used to derive a simple demo category for monthly aggregation.
Quantity	May include purchases and returns.	Used in revenue logic and return labeling.
UnitPrice	Price per item.	Combined with Quantity to form Revenue.
Country	Country of the order.	Used in daily-country revenue output.
Revenue	Not reliable until explicitly derived.	Created by the code and then summed into business-ready outputs.

Use Case 2 — how Notebook 2 works internally

Cleaning flow: what each dependency does, why the step exists, and what result it creates

Setup + S3 read

boto3 reads the Use Case 1 file from S3.
Result: one cloud-ready DataFrame.



Date standardization

Why: reporting needs usable date fields.
Result: InvoiceDate, TransactionDate, Year, Month.



Missing / invalid handling

Why: bad rows distort reporting.
Result: filtered records and cleaner text fields.



Business metric creation

Why: learners must see where Revenue and IsReturn come from.
Result: a cleaned dataset ready for ETL.

Dependencies explained

pandas handles transformation clearly for teaching. boto3 keeps all file movement in AWS terms instead of local path edits.

Key teaching line

“Every transformation changes the dataset for a reason. We are not cleaning for style; we are cleaning so the ETL output is trustworthy.”

Notebook 2 output

retail_cleaned.csv written to S3 and ready for aggregation.

Use Case 2 — Notebook 3 and Glue PySpark translation

Same business logic, two teaching layers: notebook first, Glue PySpark second

Notebook 3 — pandas ETL

- A. Extract cleaned CSV from S3
- B. Transform date/category/revenue fields
- C. Aggregate daily revenue by country
- D. Aggregate monthly revenue by category
- E. Load both outputs back to S3

Best for line-by-line explanation.

Notebook 3B — Glue PySpark ETL

- A. Spark reads the same handoff file from S3
- B. Transformations happen with distributed DataFrame operations
- C. Aggregations run across the cluster
- D. Outputs are written to S3 folders

Best for showing the scalable production pattern.

How to explain Spark simply

Driver = coordinator
Executors = workers
RDD / DataFrame = the data split across workers
Advantage = speed, scale, and fault tolerance for ETL.

Live demo tip

Run the pandas notebook cells live. Open the Glue notebook to show the parallel version and point to the same output paths in S3.

Message to learners

“The logic does not change when we move to Glue. Only the execution engine changes.”

Use Case 2 — change one row, watch the ETL output change

A simple what-if example makes ETL validation much easier to understand

Original transaction

Invoice 536518
Country: France
Quantity: 24
UnitPrice: 8.43
Revenue: 202.32

Changed transaction

Same invoice, but UnitPrice becomes 10.00
New Revenue: 240.00
Difference: +37.68

What changes in the output?

France daily revenue increases. The monthly category total also changes. If the value is invalid, it may indicate a pricing issue that needs review.

Teaching point

ETL is not only about producing files. It is about producing outputs that still make business sense when source values change.

Decision-making point

Use this slide to ask the class: “Would you trust the dashboard immediately, or would you validate the source change first?”